

# Advanced Programming

Git fundamentals



# What is Git?

Git is a free and open source **distributed version control system (VCS)** that helps track changes in source code during software development. It allows multiple developers to collaborate efficiently by keeping a history of changes, enabling branching, merging, and reverting to previous versions if needed.

# Key features of Git

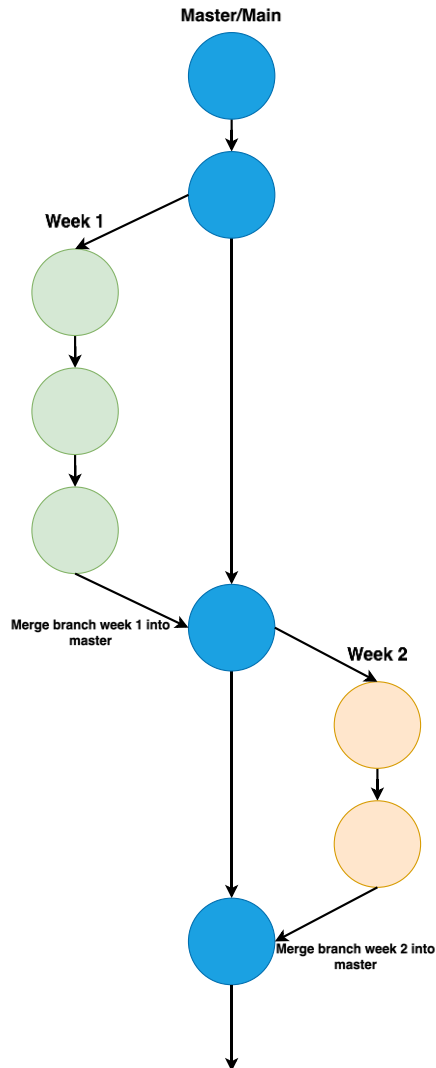
- **Distributed:** Every developer has a full copy of the repository, not just a local version of the latest files.
- **Version Control:** Tracks changes and allows reverting to previous states.
- **Branching & Merging:** Developers can work on separate branches and merge changes without affecting the main codebase.
- **Collaboration:** Makes it easy to work on projects with teams using platforms like GitHub, GitLab, or Bitbucket.
- **Efficiency:** Git is fast and works well with large projects.

# Git repository

A **Git repository (repo)** is a folder where Git tracks and manages changes to files. It contains all the necessary metadata, including commit history, branches, and version control information.

- **Local Repository** – The copy of the project on your local machine.
- **Remote Repository** – A version of the project hosted on a server (e.g., GitHub, GitLab, Bitbucket).

# Git branches & commits



A **Git commit** (represented by the circles) is a snapshot of your project at a specific point in time. It records changes made to files and saves them in the repository's history.

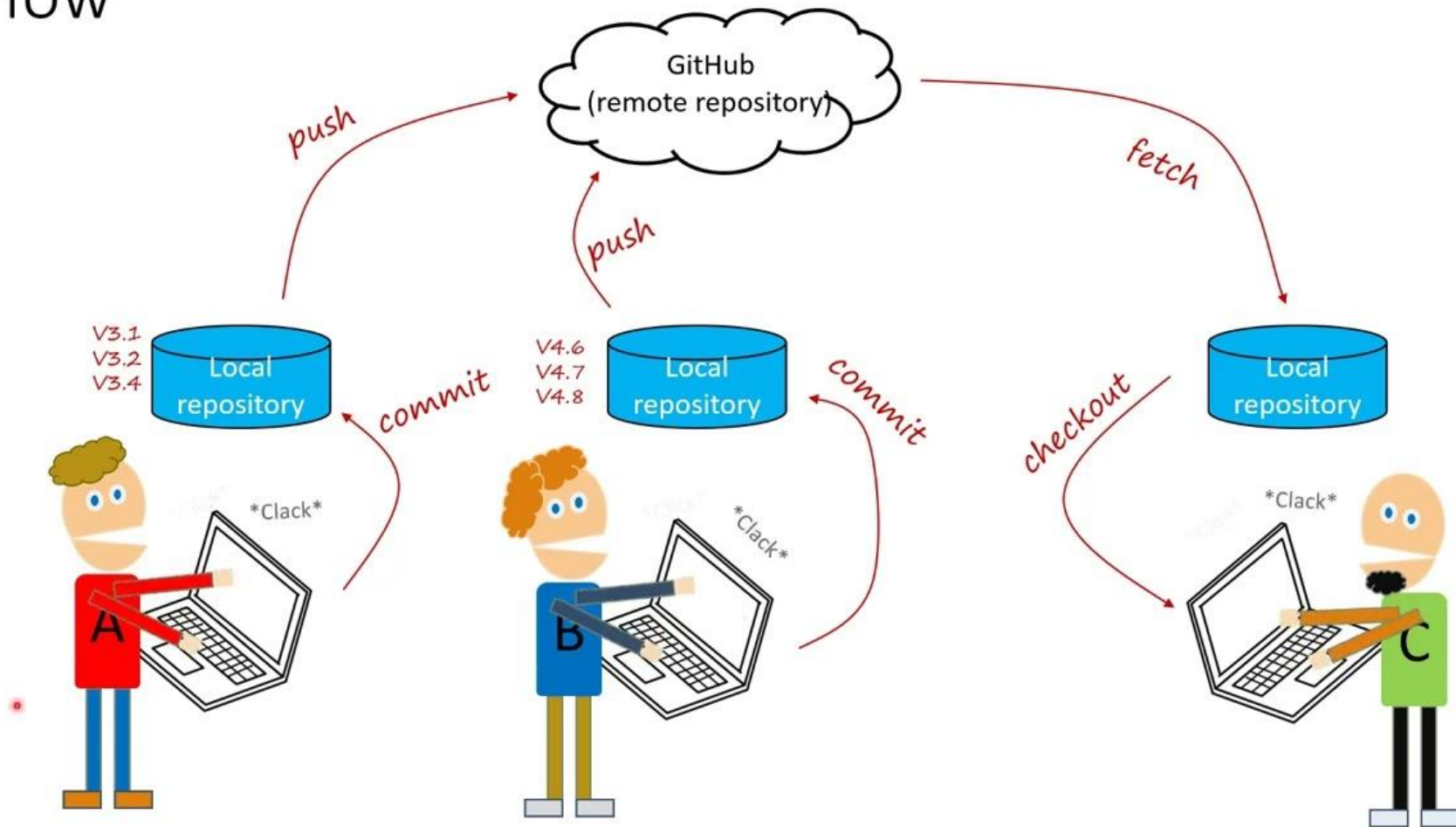
Each commit has:

- A **unique identifier (hash)** – Helps track changes.
- A **commit message** – Describes the changes.
- **Metadata** – Includes the author, timestamp, and parent commit(s).

A **Git branch** is a separate line of development in a repository. It allows multiple people (or even yourself) to work on different features, fixes, or experiments **without affecting the main codebase**.

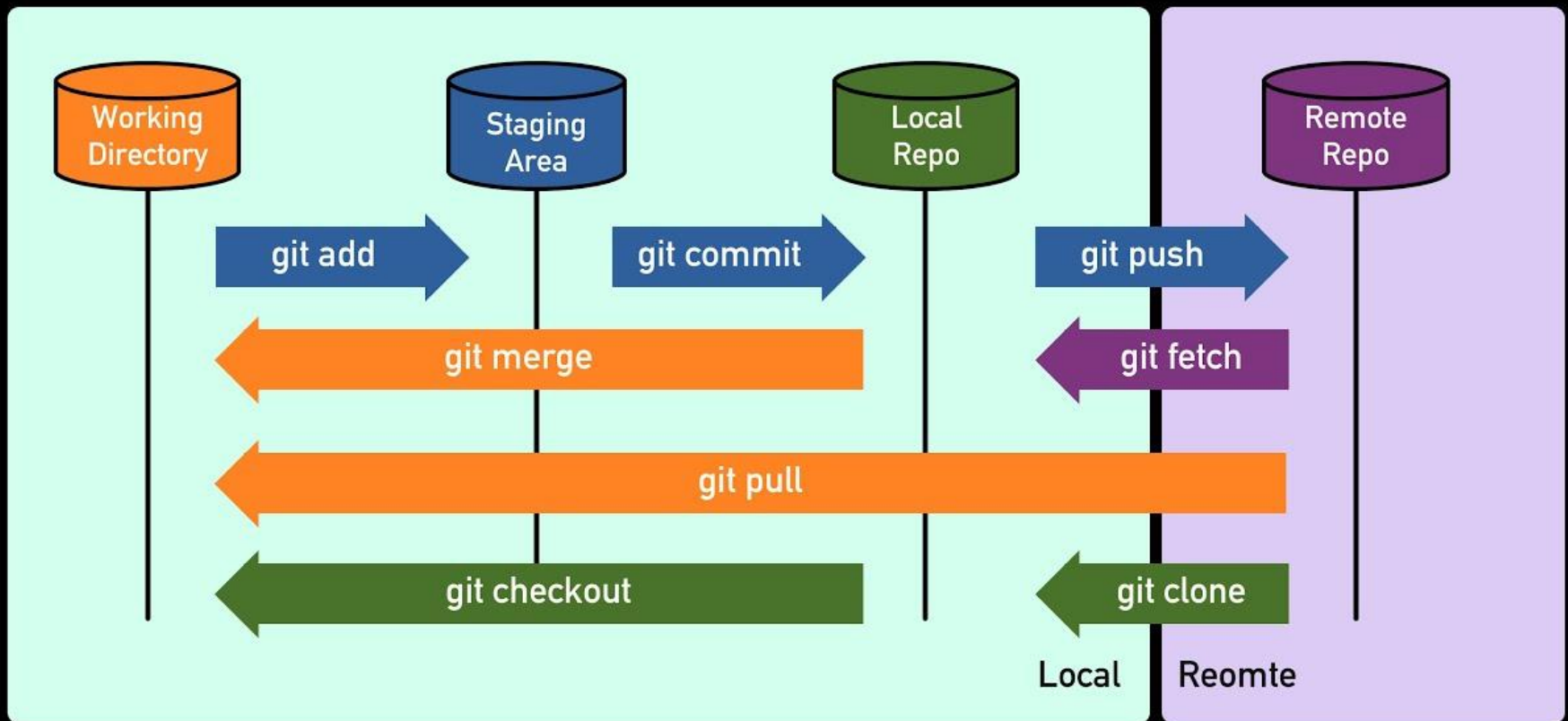
# Local & remote repositories

## How



# How does this all work?

## How Git Actually Works



# Git basic commands

- **git init** – Initializes a new Git repository.
- **git clone <repo-url>** – Clones a repository from a remote source.
- **git add <file>** – Stages changes for commit.
- **git commit -m "message"** – Saves changes with a message.
- **git push** – Uploads commits to a remote repository.
- **git pull** – Fetches updates from a remote repository.
- **git status** – Shows the current state of the working directory.
- **git branch** – Lists, creates, or deletes branches.
- **git checkout <branch>** – Switches to another branch.
- **git merge <branch>** – Merges a branch into the current one.



# Github & IntelliJ integration

The screenshot shows the IntelliJ IDEA Settings window with the 'Plugins' tab selected. The 'Marketplace' sub-tab is active, displaying search results for 'github'. The search bar at the top of the marketplace contains 'github'. Below the search bar, there are 872 search results, sorted by relevance. The first result, 'GitHub', is highlighted and has a checkmark icon. It is developed by JetBrains s.r.o. and has a version of 243.24978.79-ultimate. To the right of the search results, the details for the 'GitHub' plugin are shown. It includes the developer 'JetBrains s.r.o.', a link to the 'Plugin homepage', and a 'Disable' button. Below this, there are tabs for 'Overview', 'What's New', 'Reviews', and 'Additional Info'. The 'Overview' tab is selected, showing a description of the plugin: 'Provides integration with GitHub.' followed by a list of features: 'Specify your GitHub account to access all of your repositories from the IDE.', 'Clone repositories, create and review pull requests, create gists, and much more.', and 'Create GitHub actions and workflows'. It also states 'To configure, open Settings / Preferences and go to Version Control | GitHub.' and 'Requires the Git plugin.' At the bottom of the settings window, there are 'Cancel', 'Apply', and 'OK' buttons.

| Plugin Name            | Developer        | Download Size | Rating | Version               | Action  |
|------------------------|------------------|---------------|--------|-----------------------|---------|
| GitHub                 | JetBrains s.r.o. | 4.5M          | 3.45   | 243.24978.79-ultimate | ✓       |
| GitHub Theme           | Bhushan Gaikwad  | 537.7K        | 4.80   |                       | Install |
| GitHub Actions Manager | D Software Inc   | 299.1K        | 4.80   |                       | Install |
| GitHub Dark Theme      | Toby Johnson     | 172.5K        | 4.89   |                       | Install |
| GitHub Copilot         | GitHub           | 14.3M         | 2.50   |                       | Install |
| GitHub Color Scheme    | JetBrains s.r.o. | 763.9K        |        |                       | Install |
| GitHub 3 Color Scheme  | Sergey Makinen   | 53.4K         | 4.72   |                       | Install |
| Github-Workflow        | Yuna Morgenstern | 54K           | 4.82   |                       | Install |
| GitHub Markdown Emojis | 4ch1m            | 23.5K         | 4.23   |                       | Install |

# Useful resources

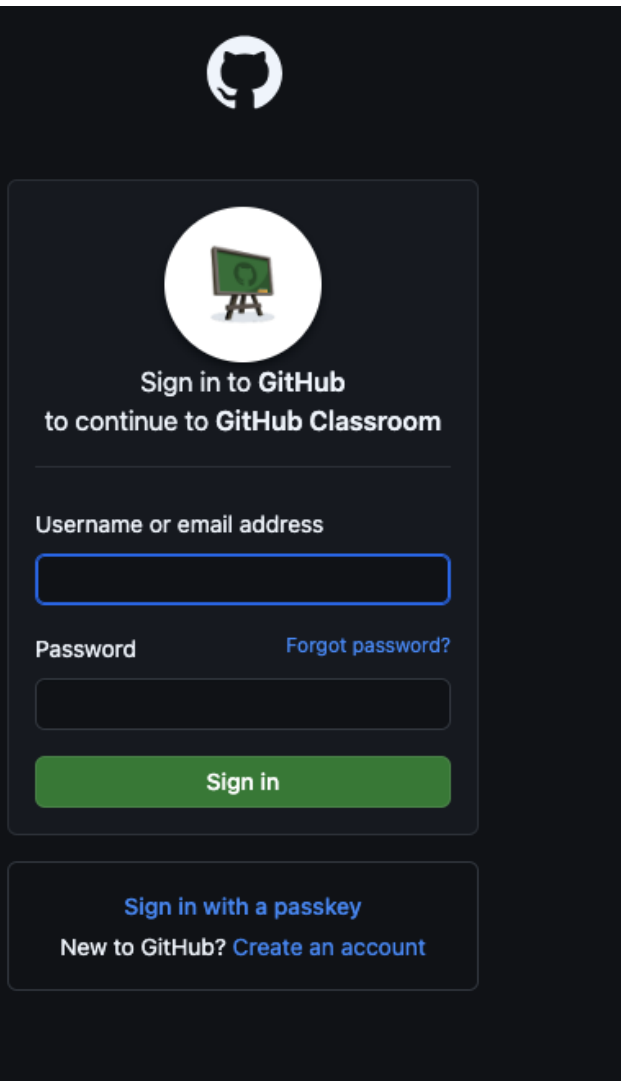
- [Git Book](#)
- [Github plugin on IntelliJ documentation](#)
- [IntelliJ IDEA youtube channel](#)

# Advanced Programming – Work assignment

First week  
Github Classroom repository setup



# Repository setup



The screenshot shows the GitHub Classroom sign-in page. At the top is the GitHub logo. Below it is a circular profile picture placeholder with a computer monitor icon. The text reads: "Sign in to GitHub to continue to GitHub Classroom". There are two input fields: "Username or email address" and "Password". A "Forgot password?" link is next to the password field. A green "Sign in" button is below the fields. At the bottom, there is a link "Sign in with a passkey" and a link "New to GitHub? Create an account".

- The teacher will share an invitation link to the Github classroom
- Students will access the link and log in with their Github accounts
  - Initially only one person of each group joins to create the team and then the other students in the group will join

# Join the classroom (1 student per group)

- Select your name from the list to join the classroom
- Only students from the corresponding class will show up
- If you can't find your name in the list, report to the teacher

Join the classroom:

**Turma P1**

To join the GitHub Classroom for this course, please select yourself from the list below to associate your GitHub account with your school's identifier (i.e., your name, ID, or email).

Can't find your name? [Skip to the next step →](#)

| Identifiers |   |
|-------------|---|
| Afonso      | > |
| Ana         | > |
| Carolina    | > |
| Henrique    | > |
| João        | > |

# Creating the team (1 student per group)

Turma P1

## Accept the group assignment — Chess Game

Before you can accept this assignment, you must create or join a team. Be sure to select the correct team as you won't be able to change this later.

Join an existing team:

|                    |      |                   |      |
|--------------------|------|-------------------|------|
| Group 1 0 students | Join | Group2 0 students | Join |
|--------------------|------|-------------------|------|

Or... Create a new team:

+ Create team

- The first student of each group to register will create the team with the provided group name

# Accept assignment (1 student per group)

- After accepting the assignment, the success screen will be shown
- Access the repository link and add it to your bookmarks
- Now, the rest of the students in the group can start registering



You're ready to go —

**Group 3**

You accepted the assignment, **Chess Game**.

Your team's assignment repository has been created:

 <https://github.com/IsecPATestOrg/chess-game-group-3>

We've configured the repository associated with this assignment.

 Your assignment is due by **Jun 1, 2025, 17:26 UTC**

Note: You may receive an email invitation to join [IsecPATestOrg](https://github.com/IsecPATestOrg) on your behalf. No further action is necessary.



# Remaining students registration

Turma P1

## Accept the group assignment — Chess Game

Before you can accept this assignment, you must create or join a team. Be sure to select the correct team as you won't be able to change this later.

Join an existing team:

|                    |      |                   |      |
|--------------------|------|-------------------|------|
| Group 1 0 students | Join | Group2 0 students | Join |
|--------------------|------|-------------------|------|

Or... Create a new team:

+ Create team

- Same process as the first student of the group
- Choose your team from the existing list by clicking **Join**



# Advanced Programming – Work assignment

First week

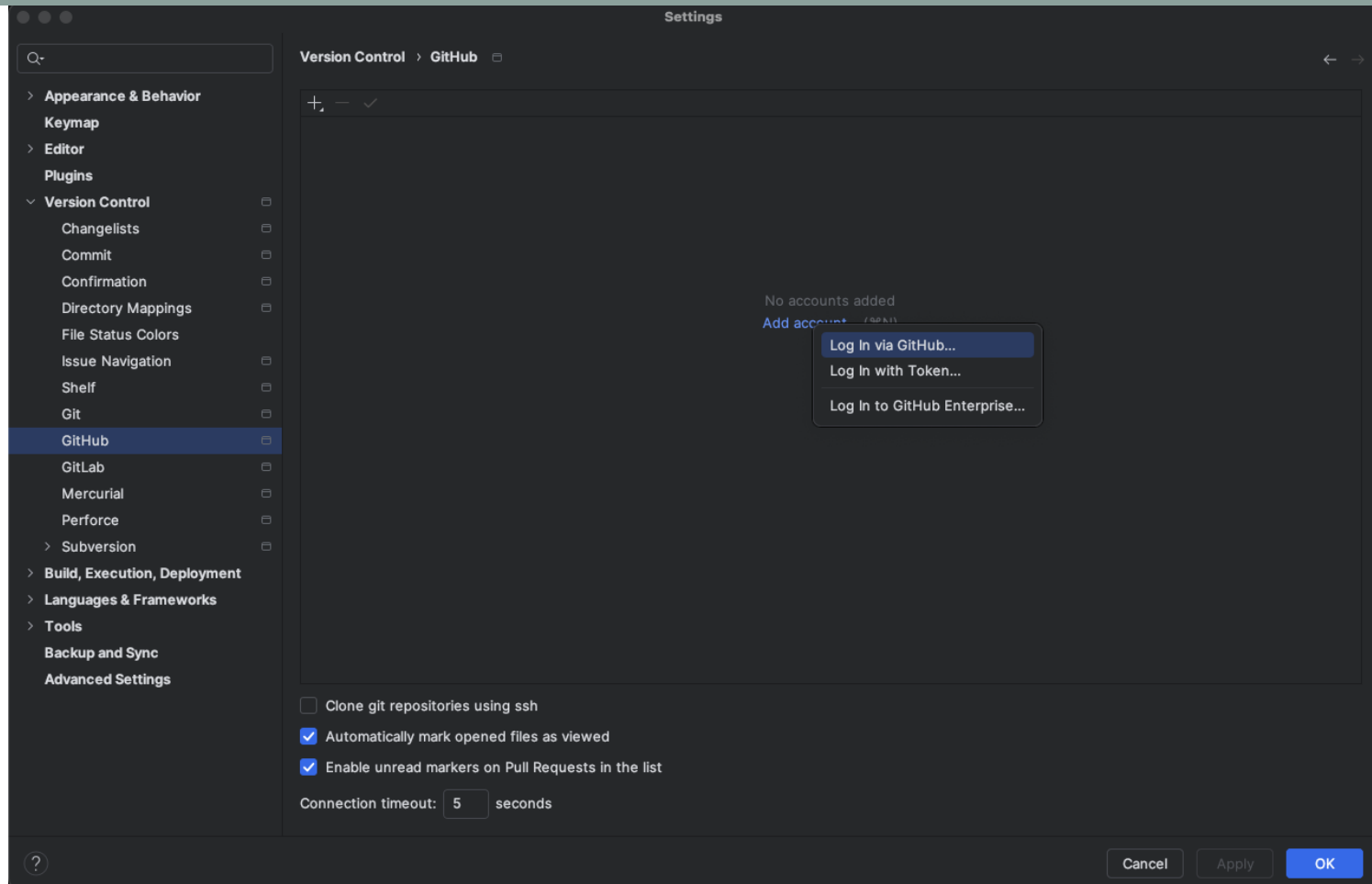
Cloning the repository in IntelliJ



**GitHub**

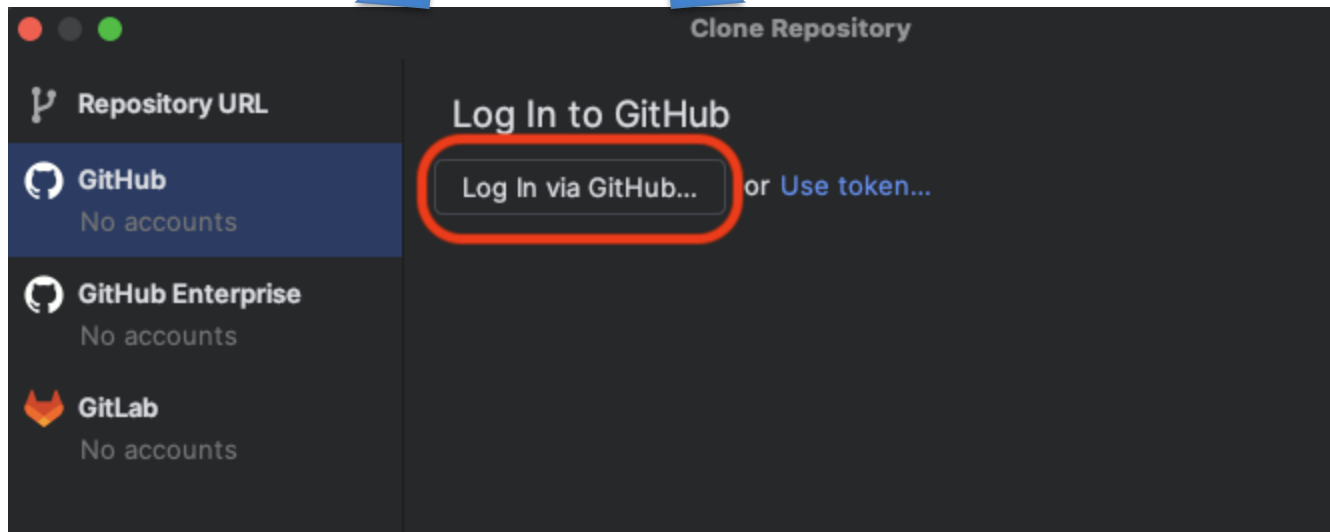
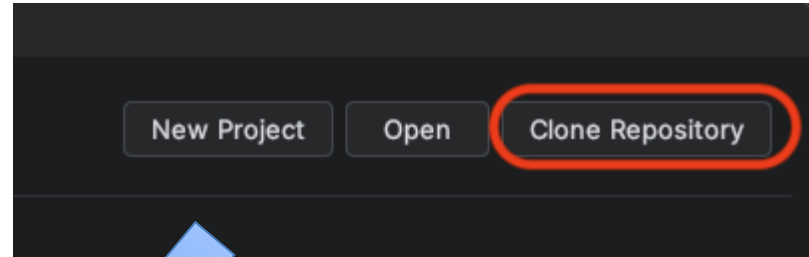
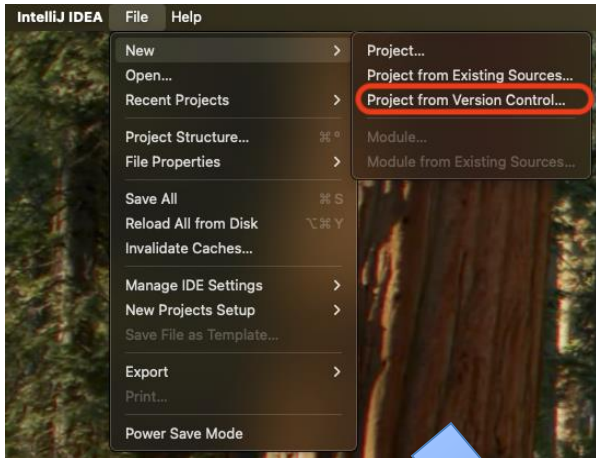


# Login into Github

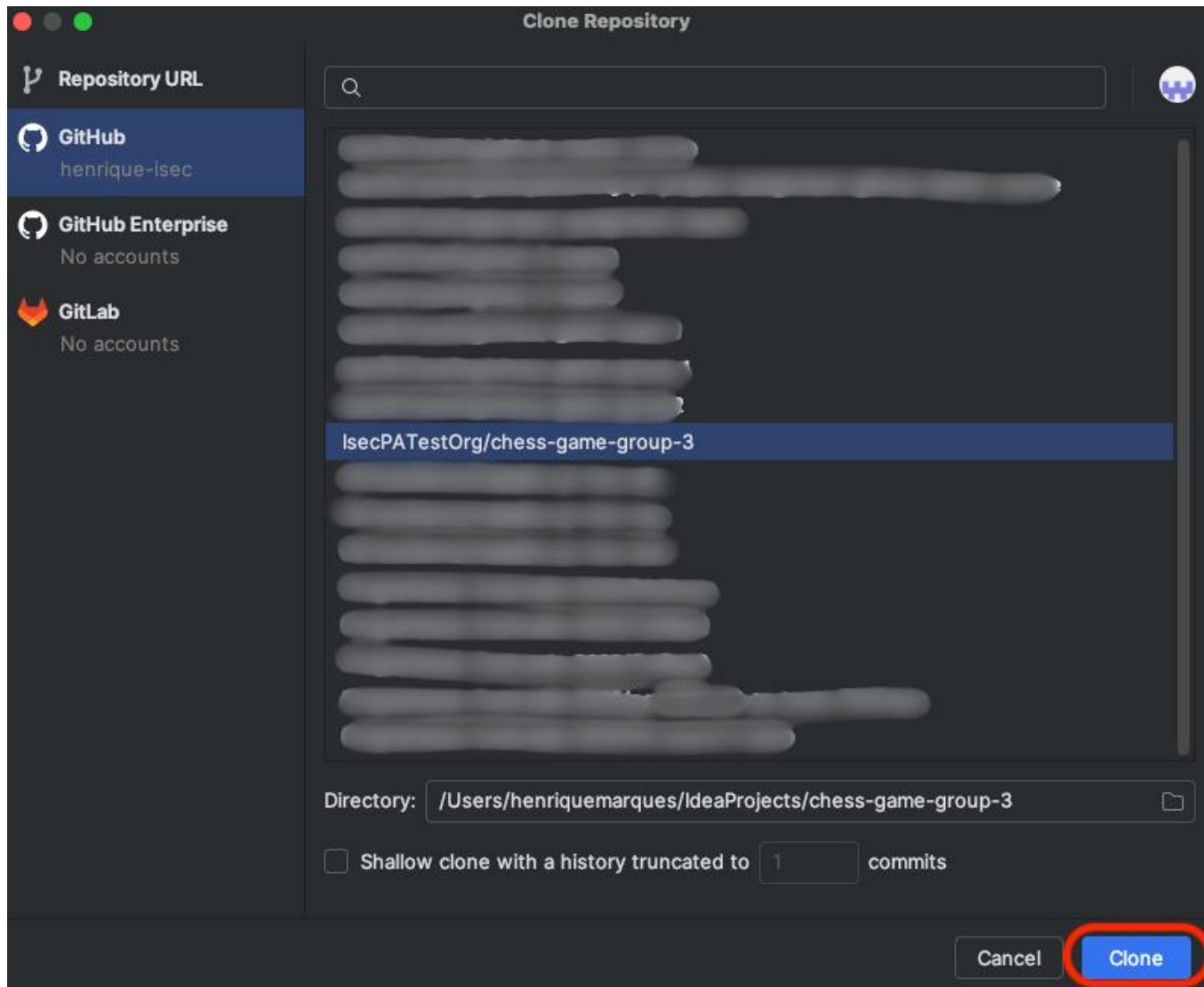


- Go into Settings > Version Control > Github > Add Account > Log In via Github
- Authorize the connection in the browser
- Confirm the account has been added and click Apply

# Login into Github (alternative)



# Clone the repository



# Advanced Programming – Work assignment

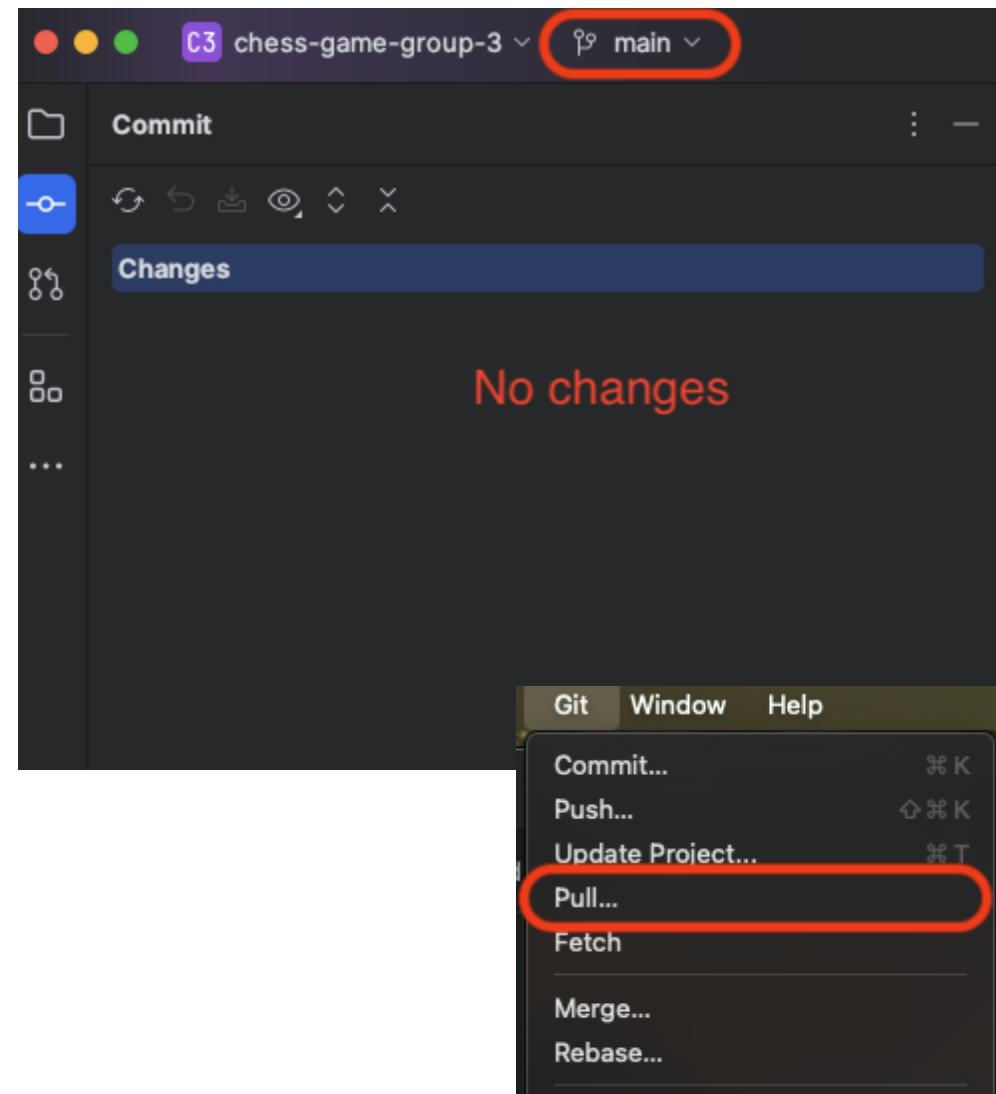
Every week  
Create a new branch



**GitHub**

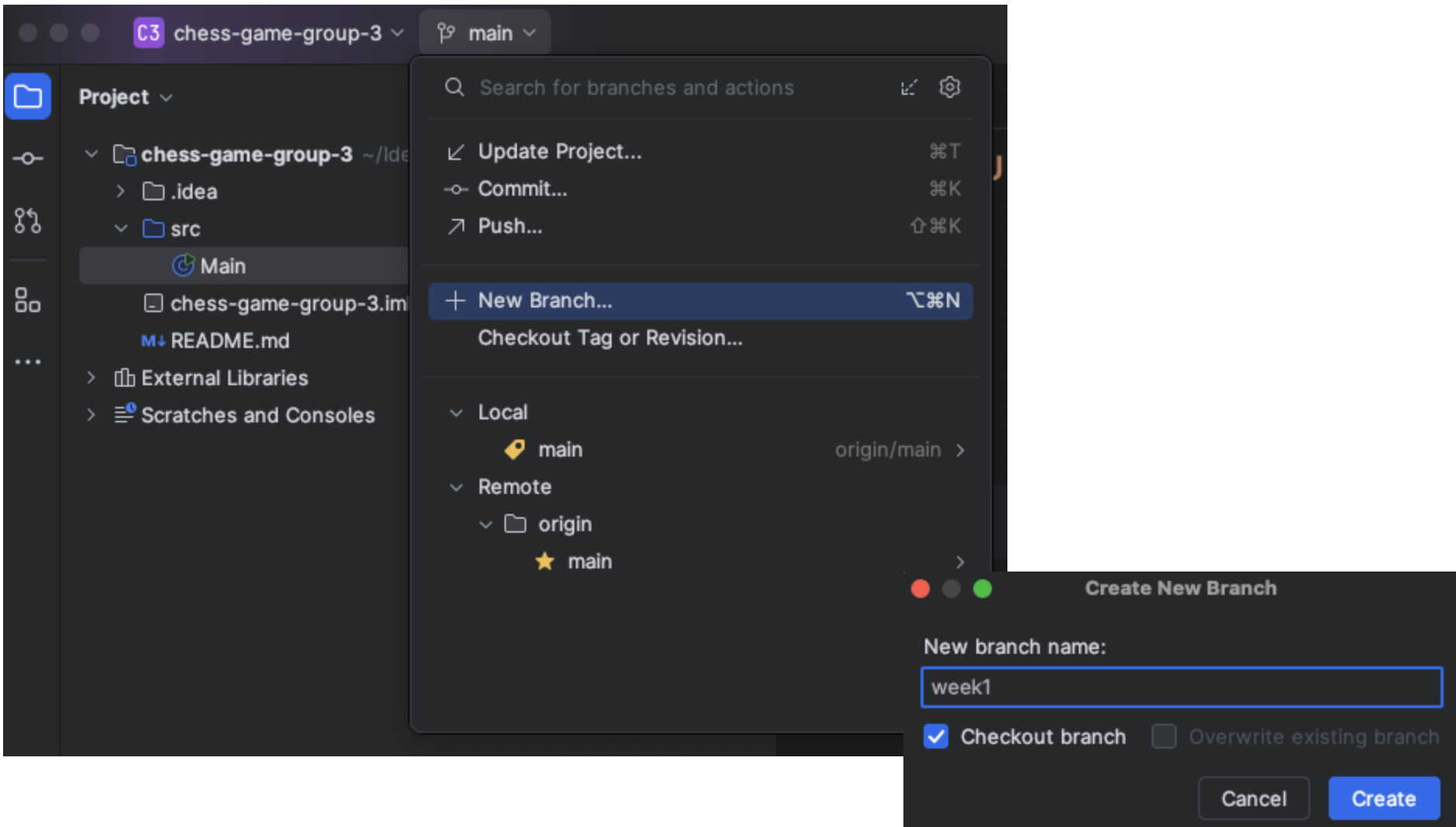


# Before create or checkout a new branch

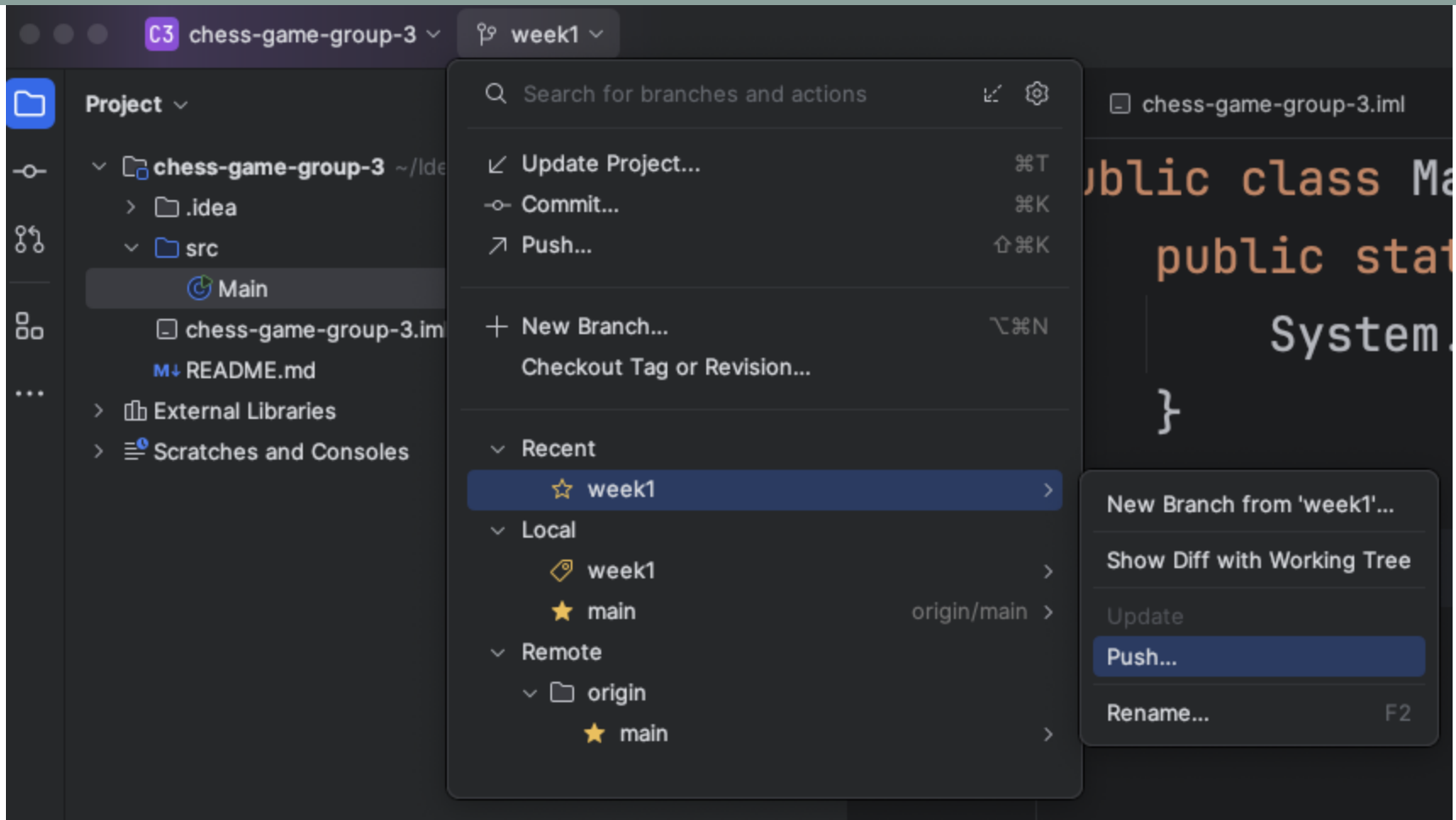


- Make sure that there are no uncommitted changes and that the current branch is main
- Update the main branch by clicking the menu **Git** and then **Pull**

# Create the new branch (1 person in the group)



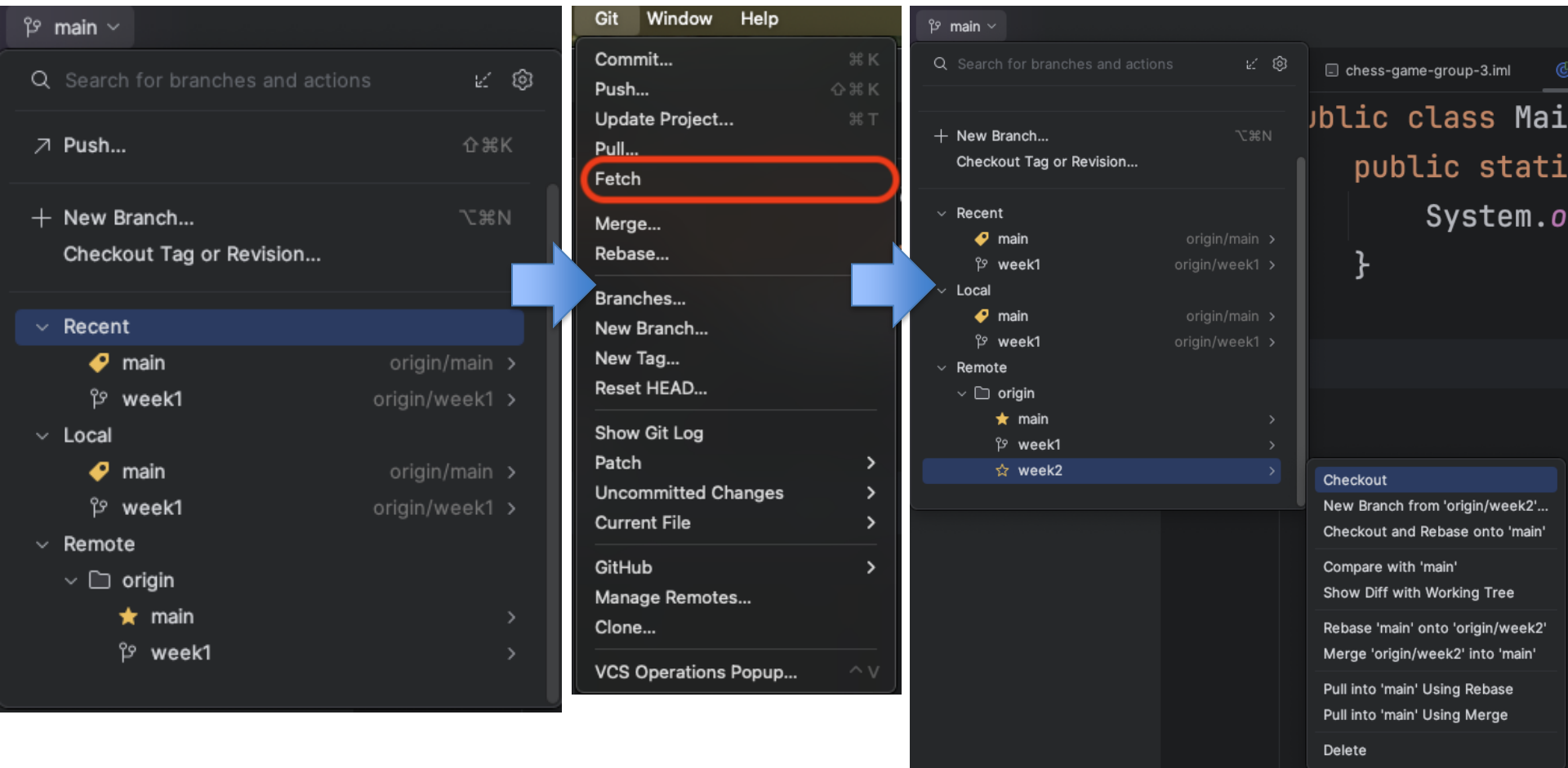
# Push the new branch (1 person in the group)



- The new branch must be pushed to be visible to other people in the group



# Checkout the new branch (other people in the group)



# Advanced Programming – Work assignment

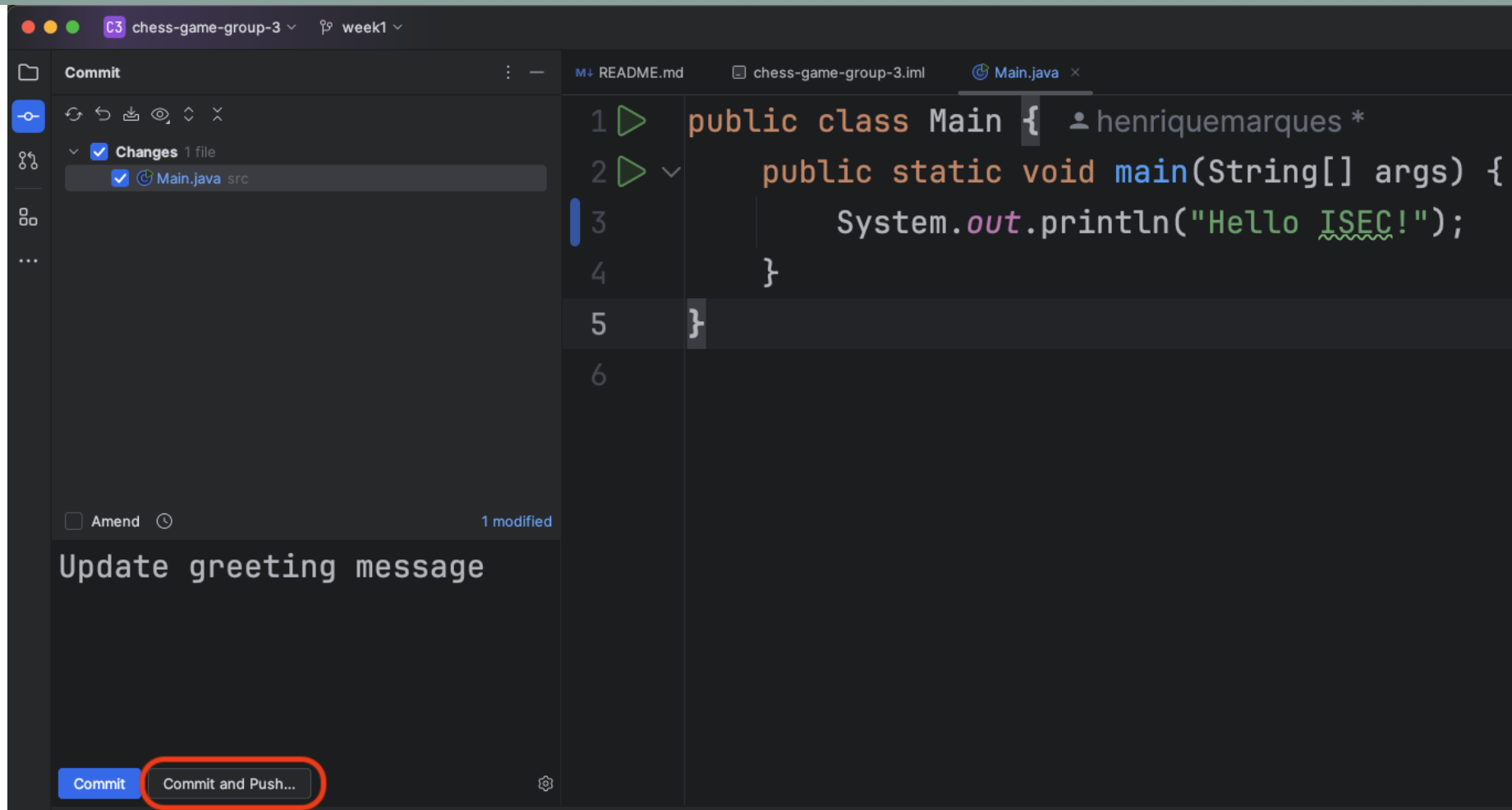
Commit & Push new code



**GitHub**

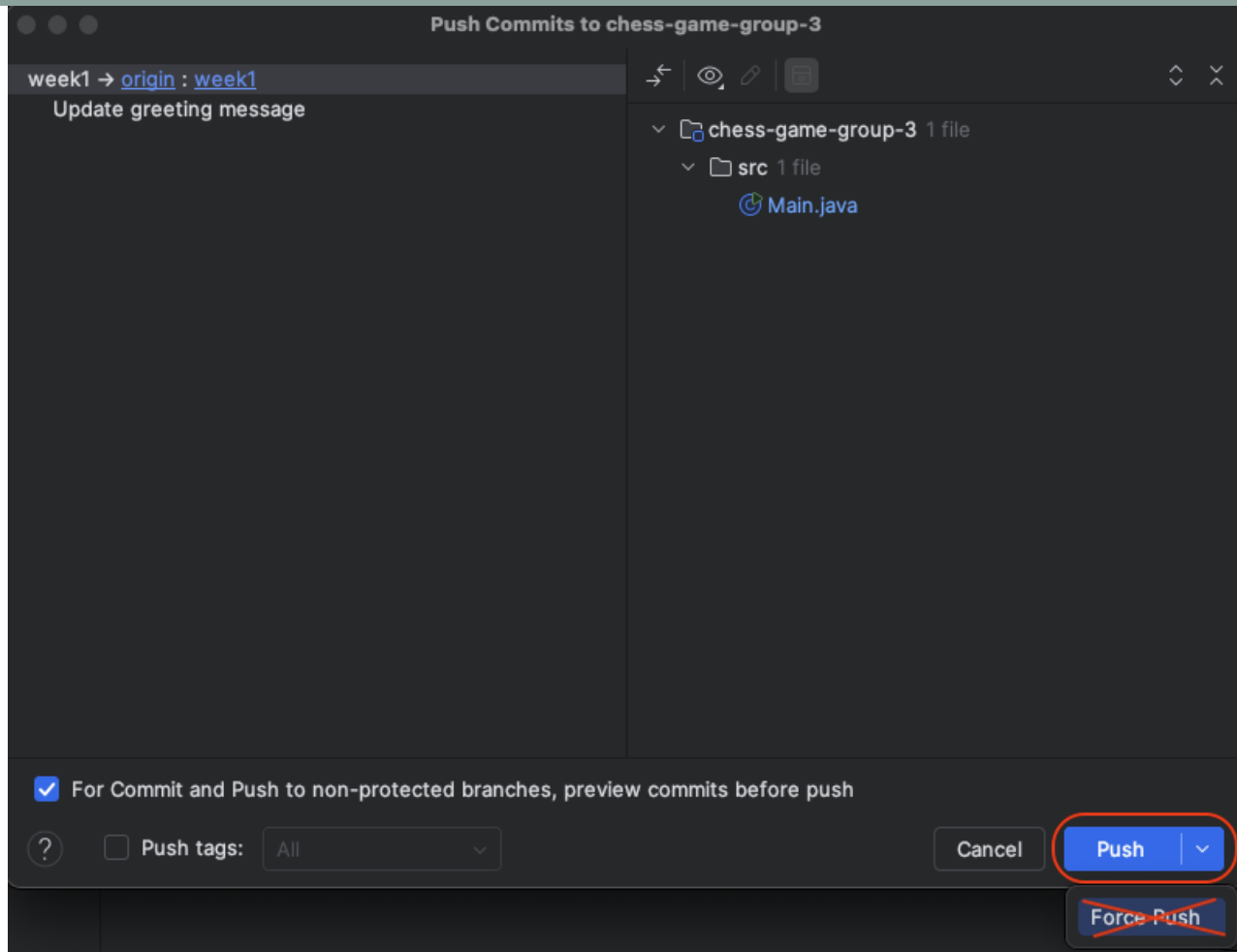


# Commit & Push new code



- Select the files you want to commit
- Write the Commit message
- Click Commit and Push

# Never force push



- **Never force push** unless you know what you are doing
- Be careful, as force push can result in code being lost

# Advanced Programming – Work assignment

Pull the new code

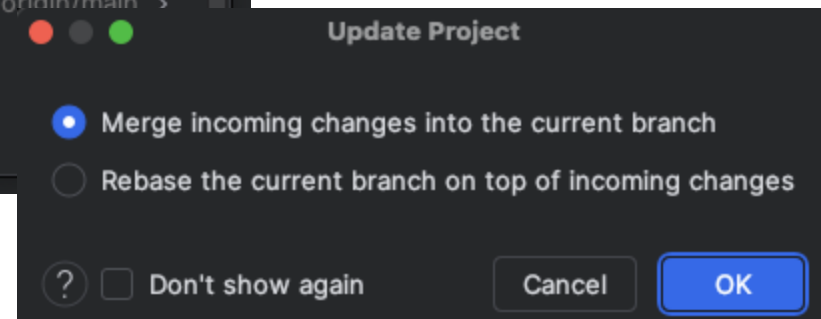
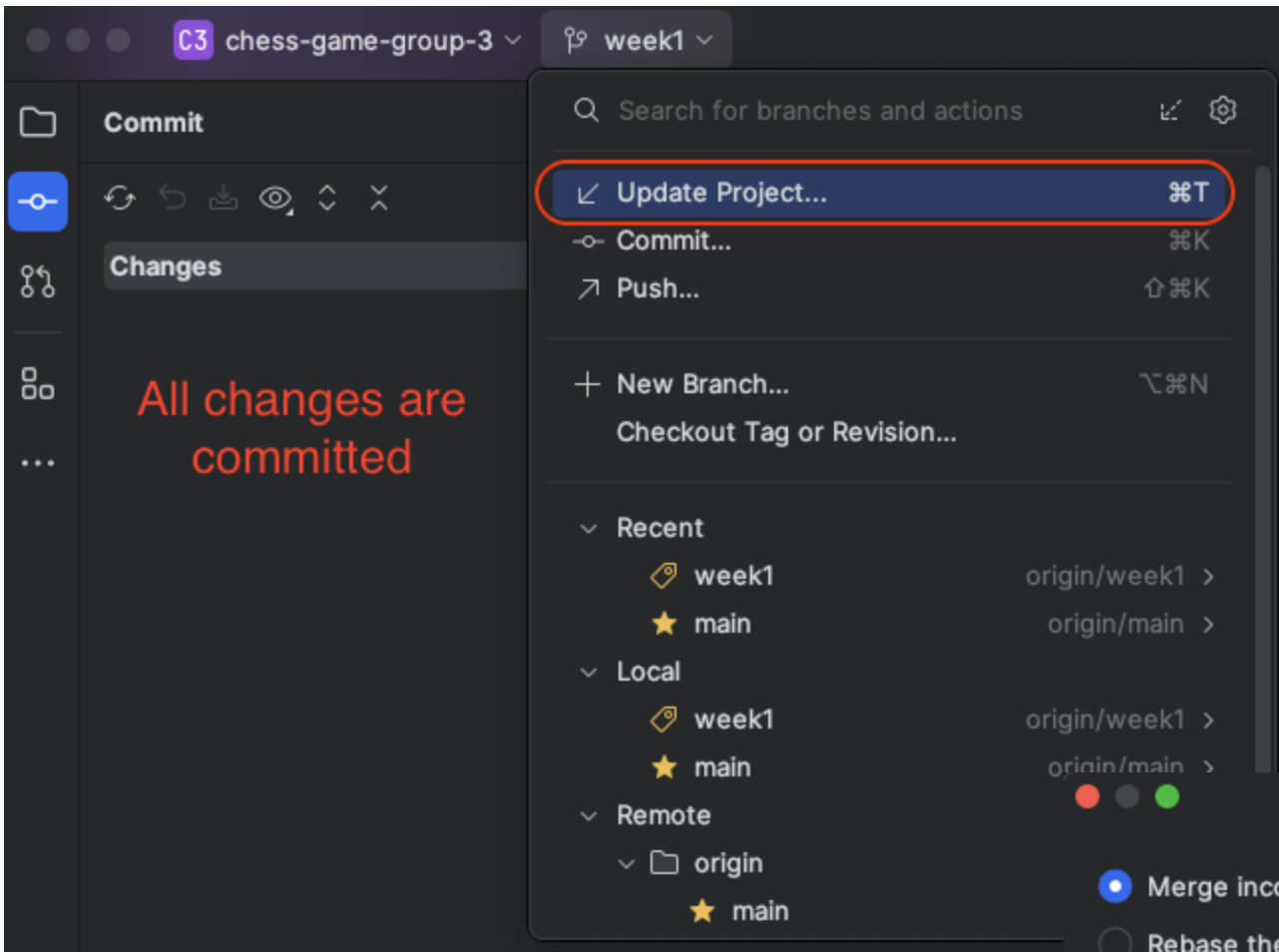


**GitHub**



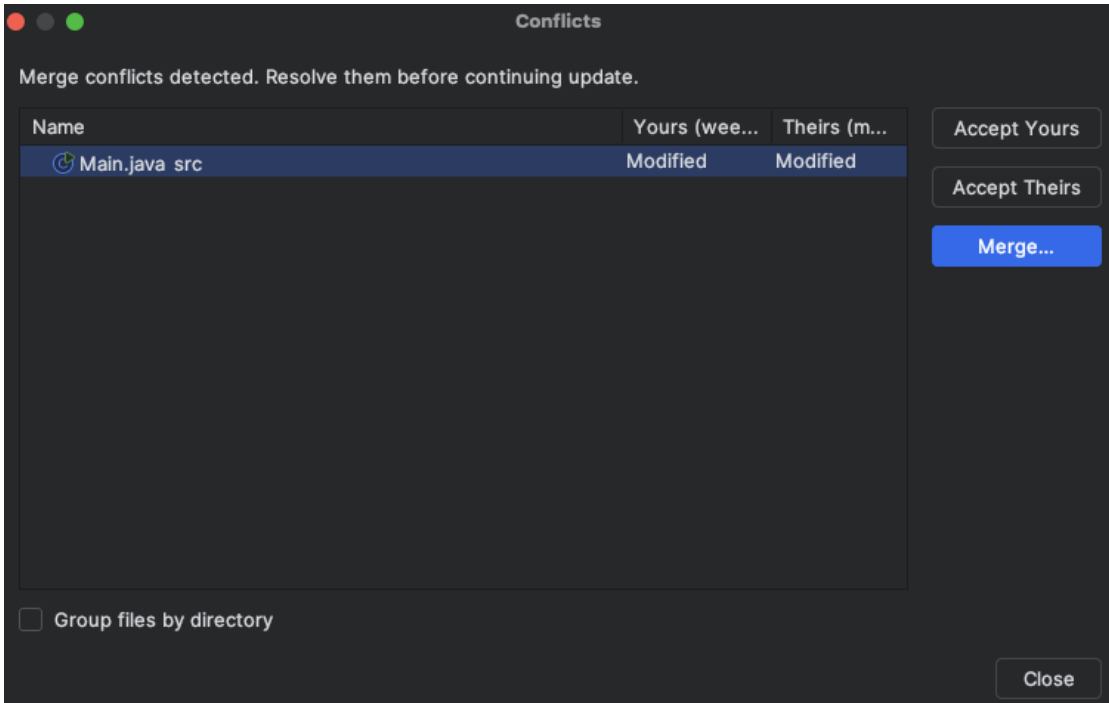
# Pulling new code

- Make sure all your code changes are committed
- Click Update Project
- Select merge incoming changes and click Ok

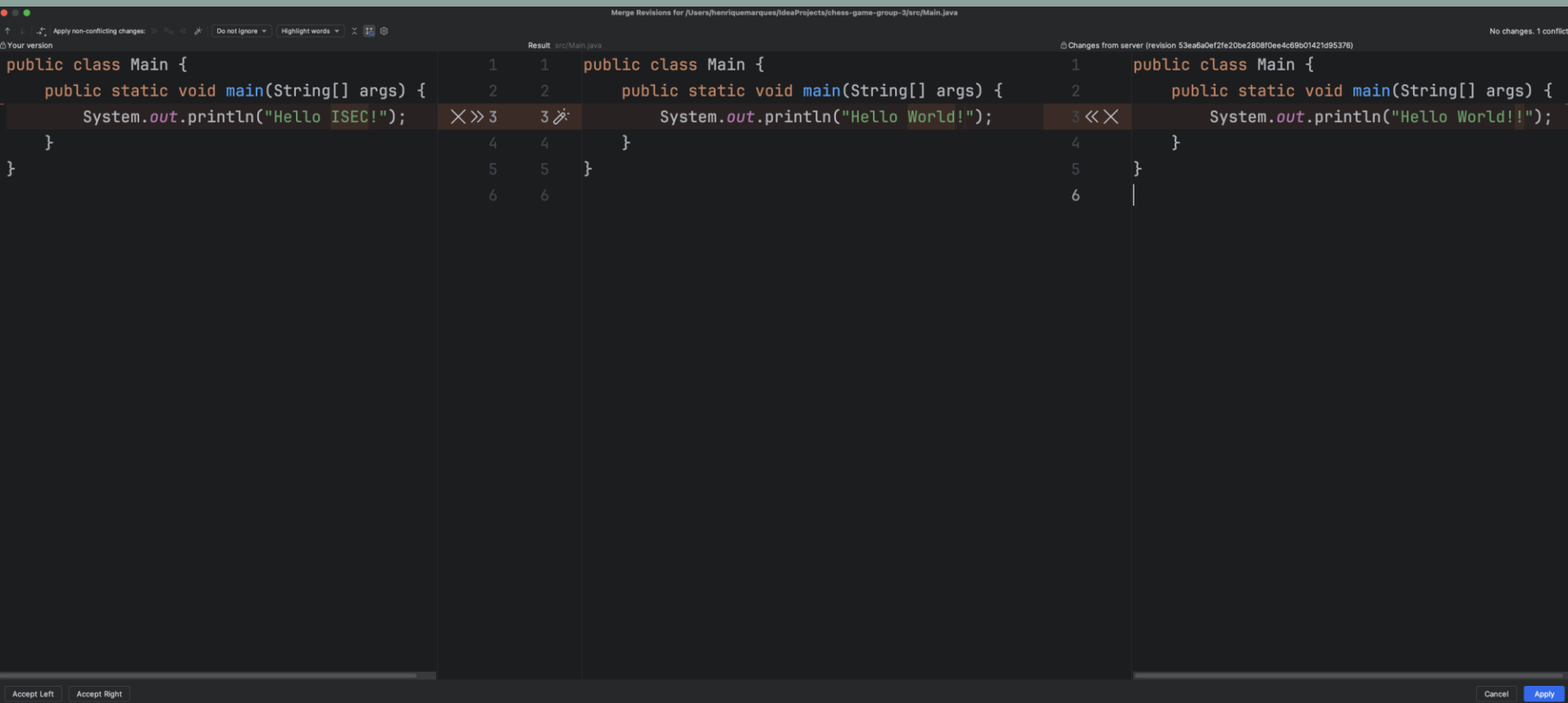


# Merge conflicts while pulling

- Merge conflicts are very common
- This happens when two people updated the same code
- Resolving the conflicts means deciding what changes should remain after the merge
- Double click on the file with conflict (Main.java) and resolve the conflicts



# Resolving merge conflicts



- In this example the two programmers updated the greeting message
- On the left are your changes and on the right the changes that were pulled
- In the middle is the result of the merge resolution
- Use the arrows in each line to choose what stays and what is discarded



# Resolving merge conflicts

Watch this 5min youtube video from JetBrains to learn how to resolve merge conflicts in IntelliJ

# Advanced Programming – Work assignment

Every week  
Create a pull request, squash & merge

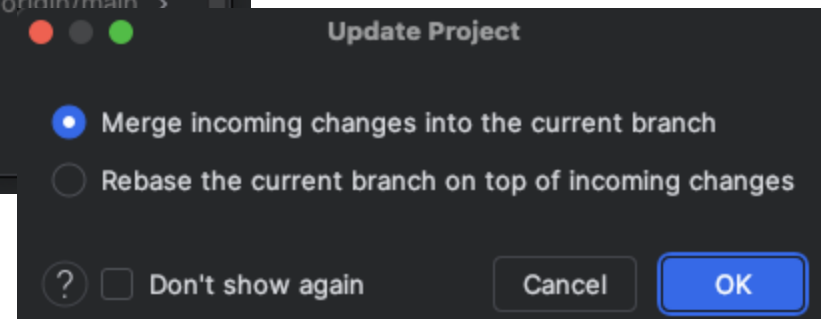
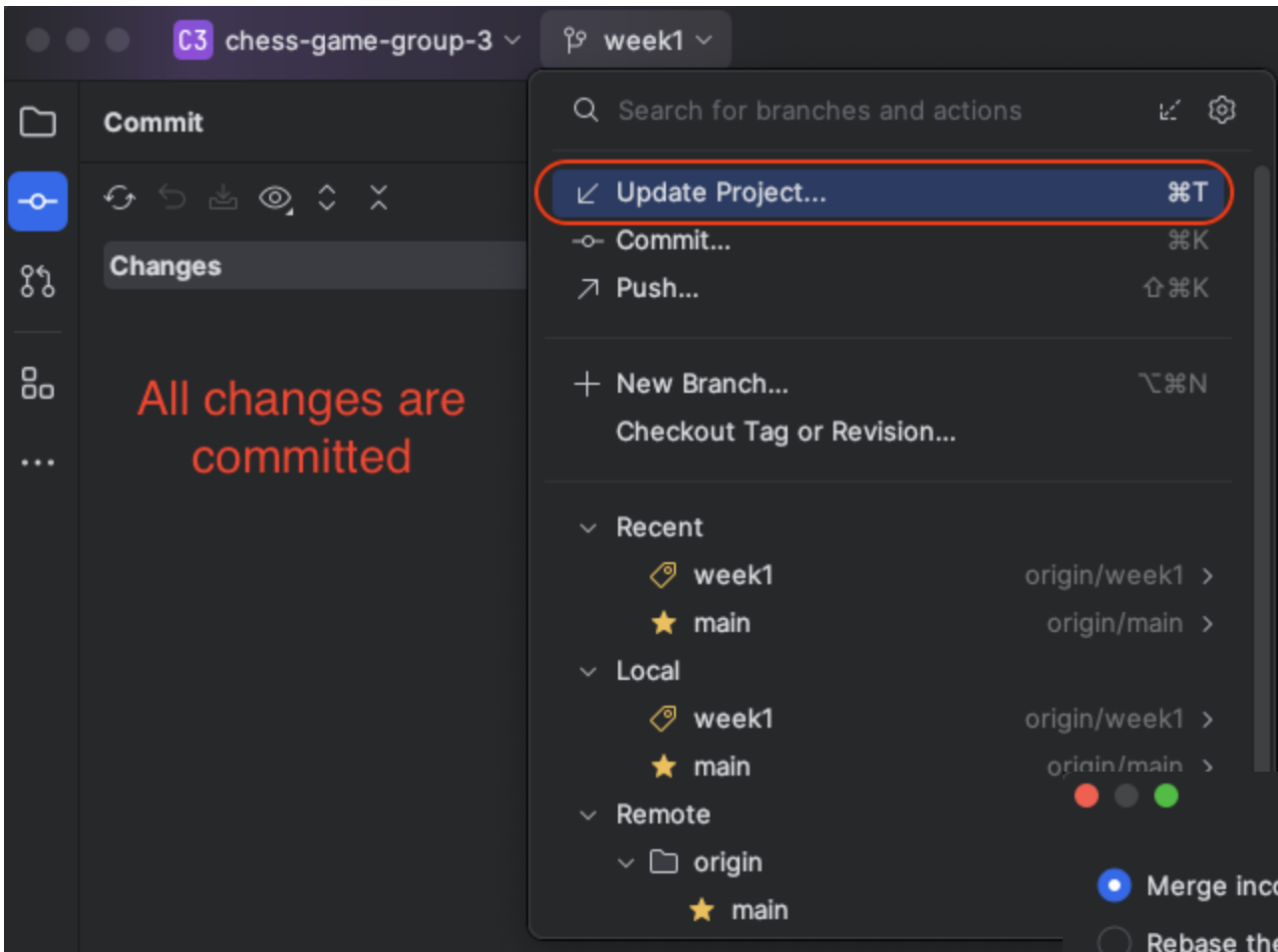


**GitHub**

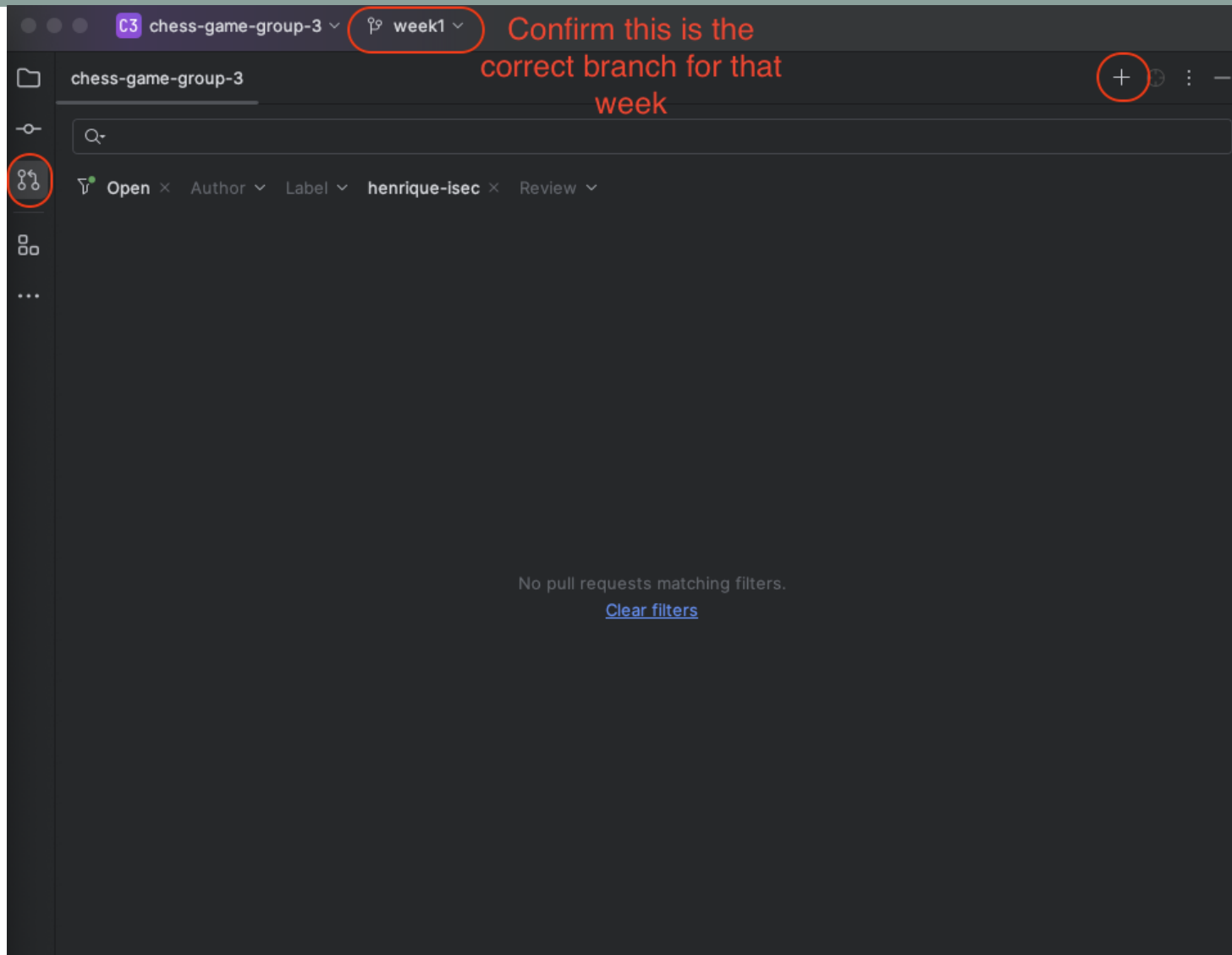


# Create a pull request

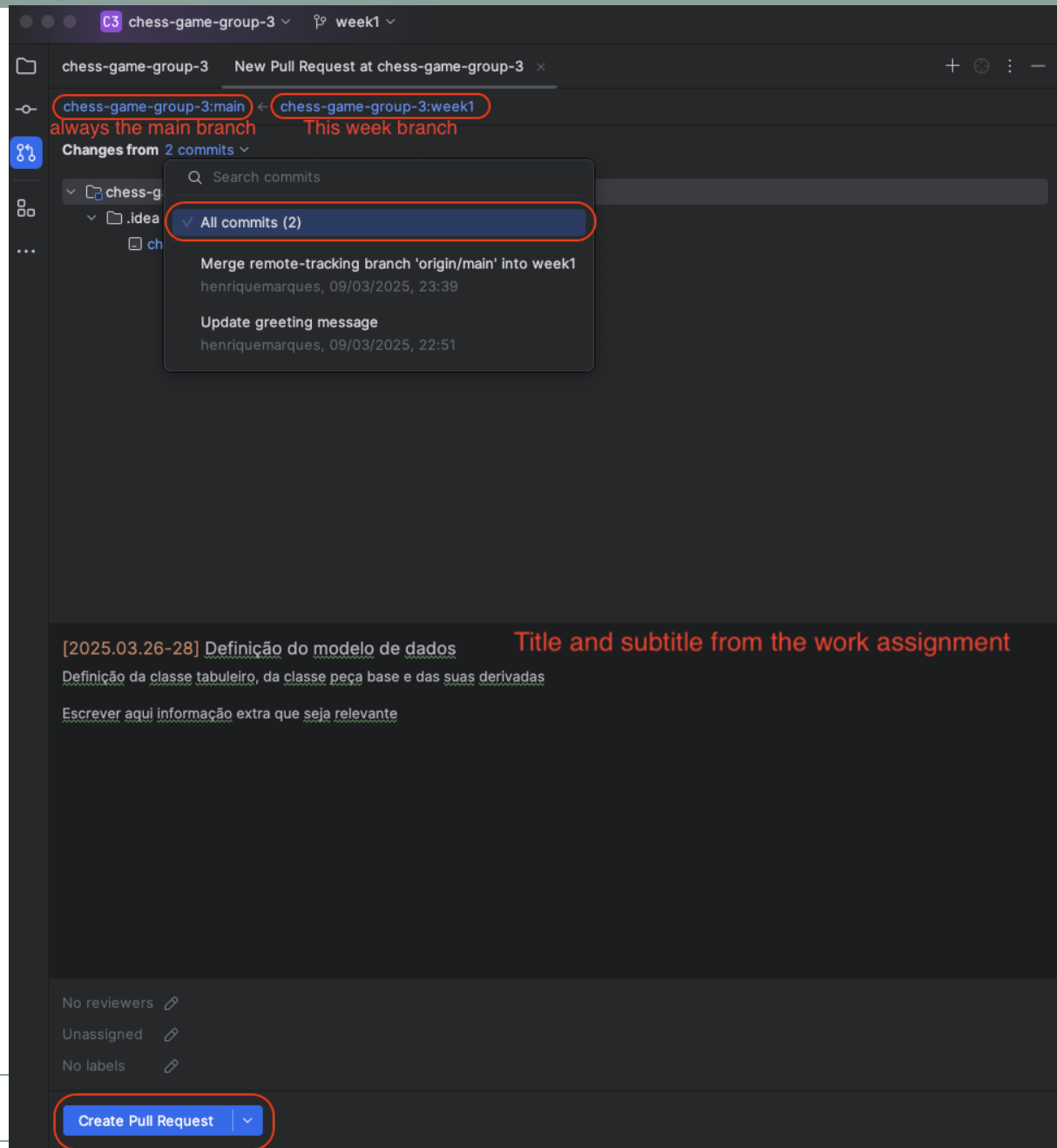
- Before creating the pull request make sure all your changes are committed, and the project is updated



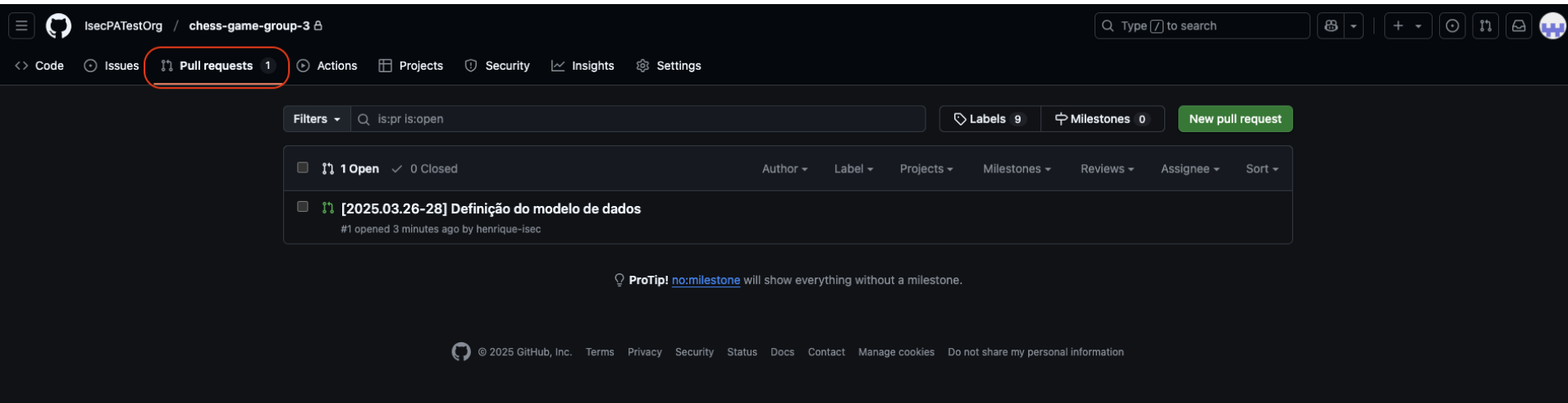
# Create a pull request



# Create a pull request



# View pull request in Github



- Go to the Pull requests tab in your Github repository

# Validate your code changes

The screenshot shows a GitHub Pull Request (PR) interface. At the top, the title is "[2025.03.26-28] Definição do modelo de dados #1". Below the title, it says "henrique-isec wants to merge 3 commits into main from week1". A red box highlights the "Files changed" tab, which shows 2 files changed (+4 -2). The main content area shows a comment from henrique-isec: "Definição da classe tabuleiro, da classe peça base e das suas derivadas. Escrever aqui informação extra que seja relevante". Below the comment, a commit history is shown: "Update greeting message", "Merge remote-tracking branch 'origin/main' into week1", and "my first pull request". A green box indicates "No conflicts with base branch" and "Merging can be performed automatically." Below this, there is a "Squash and merge" button. On the right side, there are sections for "Reviewers" (No reviews), "Assignees" (No one—assign yourself), "Labels" (None yet), "Projects" (None yet), "Milestone" (No milestone), and "Development" (Successfully merging this pull request may close these issues.). At the bottom, there is a "Add a comment" section with a text area and a "Comment" button. A "ProTip!" at the bottom suggests adding comments to specific lines under the "Files changed" tab.

# Validate your code changes

The screenshot shows a GitHub pull request for the repository 'lsecPATestOrg / chess-game-group-3'. The pull request is titled '[2025.03.26-28] Definição do modelo de dados #1' and is created by 'henrique-isec'. It shows 3 commits to be merged into the 'main' branch from the 'week1' branch. The interface includes tabs for Conversation, Commits, Checks, and Files changed. The 'Files changed' tab is active, showing a list of files: '.idea/chess-game-group-3.iml' and 'src/Main.java'. The 'src/Main.java' file is expanded, showing a diff with 2 lines added and 1 line removed. The diff shows the addition of a new print statement: 'System.out.println("My first pull request!!");'. The interface also includes a search bar, a 'Type / to search' prompt, and a 'Review changes' button. A warning message at the bottom states: 'Confirm all the files and changes are correct! If something is not correct, commit and push the changes to the same branch. This pull request will update automatically.'

lsecPATestOrg / chess-game-group-3

Code Issues Pull requests 1 Actions Projects Security Insights Settings

[2025.03.26-28] Definição do modelo de dados #1

henrique-isec wants to merge 3 commits into main from week1

Conversation 0 Commits 3 Checks 0 Files changed 2

Changes from all commits File filter Conversations

1/2 files viewed Ask Copilot Review changes

Filter changed files

.idea/chess-game-group-3.iml

src/Main.java

```
@@ -1,5 +1,5 @@
1 1 public class Main {
2 2 public static void main(String[] args) {
3 - System.out.println("Hello World!!");
3 + System.out.println("My first pull request!!");
4 4 }
5 5 }
```

Confirm all the files and changes are correct!  
If something is not correct, commit and push the changes to the same branch. This pull request will update automatically.



# Squash and merge

The screenshot shows a GitHub pull request interface. At the top, a green checkmark icon is next to the text "No conflicts with base branch" and "Merging can be performed automatically." Below this, a green button labeled "Squash and merge" has a dropdown arrow, which is circled in red. To the right of the button, it says "You can also merge this with the command line. [View command line instructions.](#)".

A dropdown menu is open, showing three options:

- Create a merge commit**  
All commits from this branch will be added to the base branch via a merge commit.
- Squash and merge** (highlighted with a red circle)  
The 3 commits from this branch will be combined into one commit in the base branch.
- Rebase and merge**  
The 3 commits from this branch will be rebased and added to the base branch.

Below the dropdown menu, there is a "Markdown is supported" icon and a "Paste, drop, or click to add files" button. At the bottom right, there are two buttons: "Close pull request" (with a red 'X' icon) and "Comment".

# Squash and merge

